KONJETI PAVAN DEEKSHITH
192412172

CO5 ASSESSMENT 2

PSEUDOCODE WRITING TASK — ANSWERS

Question 1: Real-Time Face Detection and Recognition

A security system must continuously capture a live camera stream, detect multiple faces simultaneously, recognize registered persons against a trained dataset, and display bounding boxes with identity labels — all while operating in real time with minimized processing delay. The pipeline below (image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control) addresses each requirement using an OpenCV-based approach.

Pipeline Overview Diagram

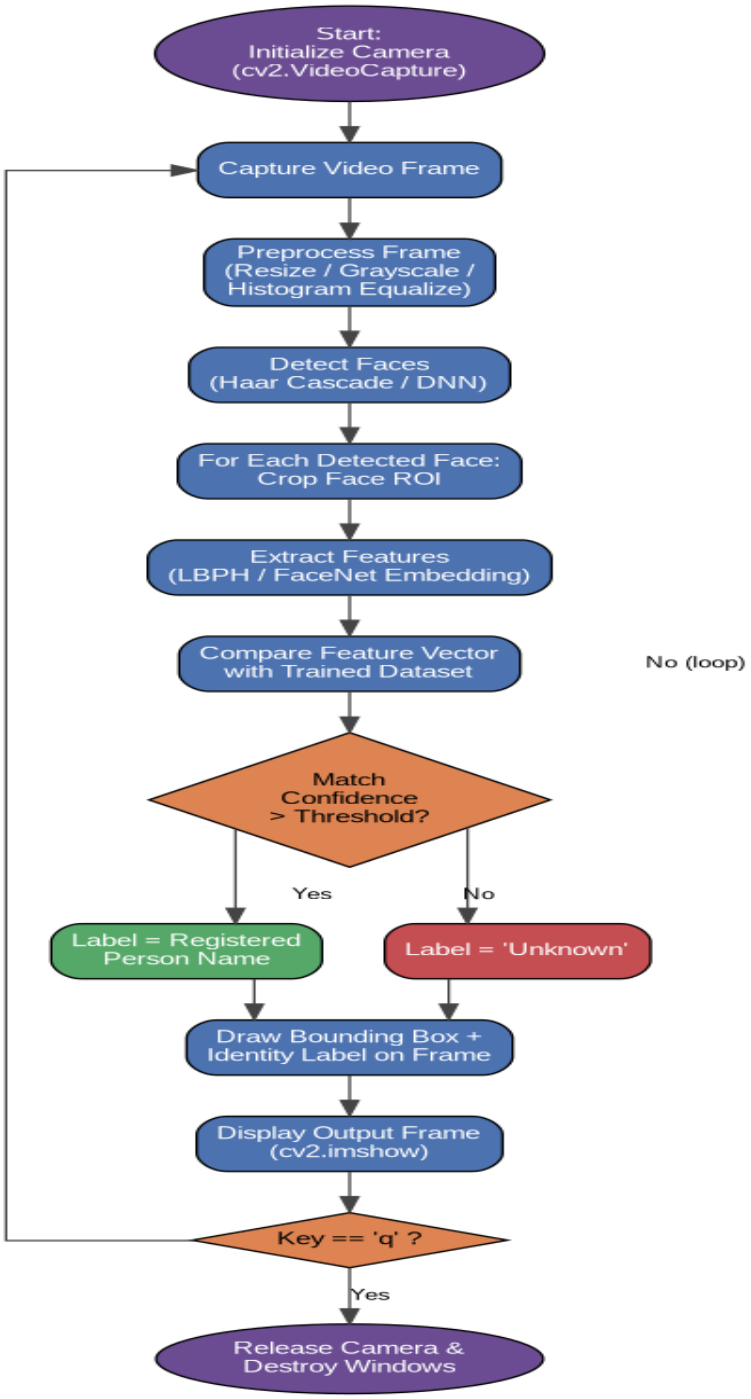

Fig. 1 — Real-Time Face Detection & Recognition Pipeline

Detailed Pseudocode

```
BEGIN FaceDetectionAndRecognitionSystem

    // ---------- 1. INITIALIZATION ----------
    LOAD trained face recognizer model (e.g., LBPH / FaceNet embeddings)
    LOAD label map: {label_id -> person_name} from registered dataset
    LOAD face detector model (Haar Cascade XML or DNN prototxt+weights)
    SET confidence_threshold = T          // recognition acceptance threshold
    SET frame_skip = N                    // optional frame-skipping for speed
    OPEN camera stream: cap = VideoCapture(source)
    IF cap NOT opened THEN
        PRINT "Error: cannot access camera" AND EXIT
    END IF
    frame_count = 0

    // ---------- 2. MAIN REAL-TIME LOOP ----------
    WHILE camera stream is active DO
        success, frame = cap.read()
        IF NOT success THEN
            BREAK   // stream ended or read error
        END IF
        frame_count = frame_count + 1

        // ---------- 3. PREPROCESSING ----------
        resized_frame = RESIZE(frame, target_width, target_height)
        gray_frame = CONVERT_TO_GRAYSCALE(resized_frame)
        gray_frame = HISTOGRAM_EQUALIZE(gray_frame)   // improves detection in poor light

        // Optional: skip heavy processing on some frames to keep real-time speed
        IF frame_skip > 0 AND (frame_count MOD frame_skip != 0) THEN
            DISPLAY(resized_frame)
            IF KEY_PRESSED('q') THEN BREAK
            CONTINUE
        END IF

        // ---------- 4. FACE DETECTION (multiple faces) ----------
        face_boxes = detector.detectMultiScale(gray_frame,
                                               scaleFactor = 1.1,
                                               minNeighbors = 5,
                                               minSize = (30, 30))
        // face_boxes = list of (x, y, w, h) for every face found

        // ---------- 5. PROCESS EACH DETECTED FACE ----------
        FOR EACH (x, y, w, h) IN face_boxes DO
            face_roi = gray_frame[y : y+h, x : x+w]
            face_roi = RESIZE(face_roi, fixed_size)   // normalize input size
```

```
            // ----- 5a. FEATURE EXTRACTION -----
            feature_vector = EXTRACT_FEATURES(face_roi)   // LBPH histogram or CNN
        embedding

            // ----- 5b. RECOGNITION -----
            predicted_id, confidence = recognizer.predict(feature_vector)

            IF confidence >= confidence_threshold THEN
                identity_label = label_map[predicted_id]
                box_color = GREEN
            ELSE
                identity_label = "Unknown"
                box_color = RED
            END IF

            // ----- 5c. VISUALIZATION -----
            DRAW_RECTANGLE(resized_frame, (x, y), (x+w, y+h), box_color, thickness=2)
            DRAW_TEXT(resized_frame, identity_label, position=(x, y-10),
                        font, box_color)
        END FOR

        // ---------- 6. DISPLAY OUTPUT ----------
        DISPLAY(resized_frame, window_name = "Face Detection & Recognition")

        // ---------- 7. LOOP CONTROL / EXIT ----------
        IF KEY_PRESSED('q') THEN
            BREAK
        END IF
    END WHILE

    // ---------- 8. CLEANUP ----------
    cap.RELEASE()
    DESTROY_ALL_WINDOWS()

END FaceDetectionAndRecognitionSystem
```

Key Design Notes

- Real-time performance: frame resizing, optional frame-skipping, and grayscale conversion reduce per-frame computation cost.
- Multiple faces handled simultaneously: detect Multiscale returns a list of all face regions in one call; each is processed independently inside a FOR loop.
- Minimized delay: histogram equalization and lightweight Haar-cascade detection (or a fast DNN detector) keep detection latency low; heavy recognition runs only on detected ROIs, not the full frame.
- Robust recognition: an unknown/low-confidence face is explicitly labeled 'Unknown' rather than forced into an incorrect match.
- Efficient loop control: the WHILE loop checks for a keypress each iteration and breaks cleanly, releasing the camera and destroying windows to avoid resource leaks.

Question 2: Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to detect vehicles entering a predefined region of interest, track multiple vehicles across consecutive frames, count vehicles crossing a virtual line without double counting, and display bounding boxes, tracking IDs, and the running vehicle count — while maintaining near real-time performance. The pipeline below covers video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

Pipeline Overview Diagram

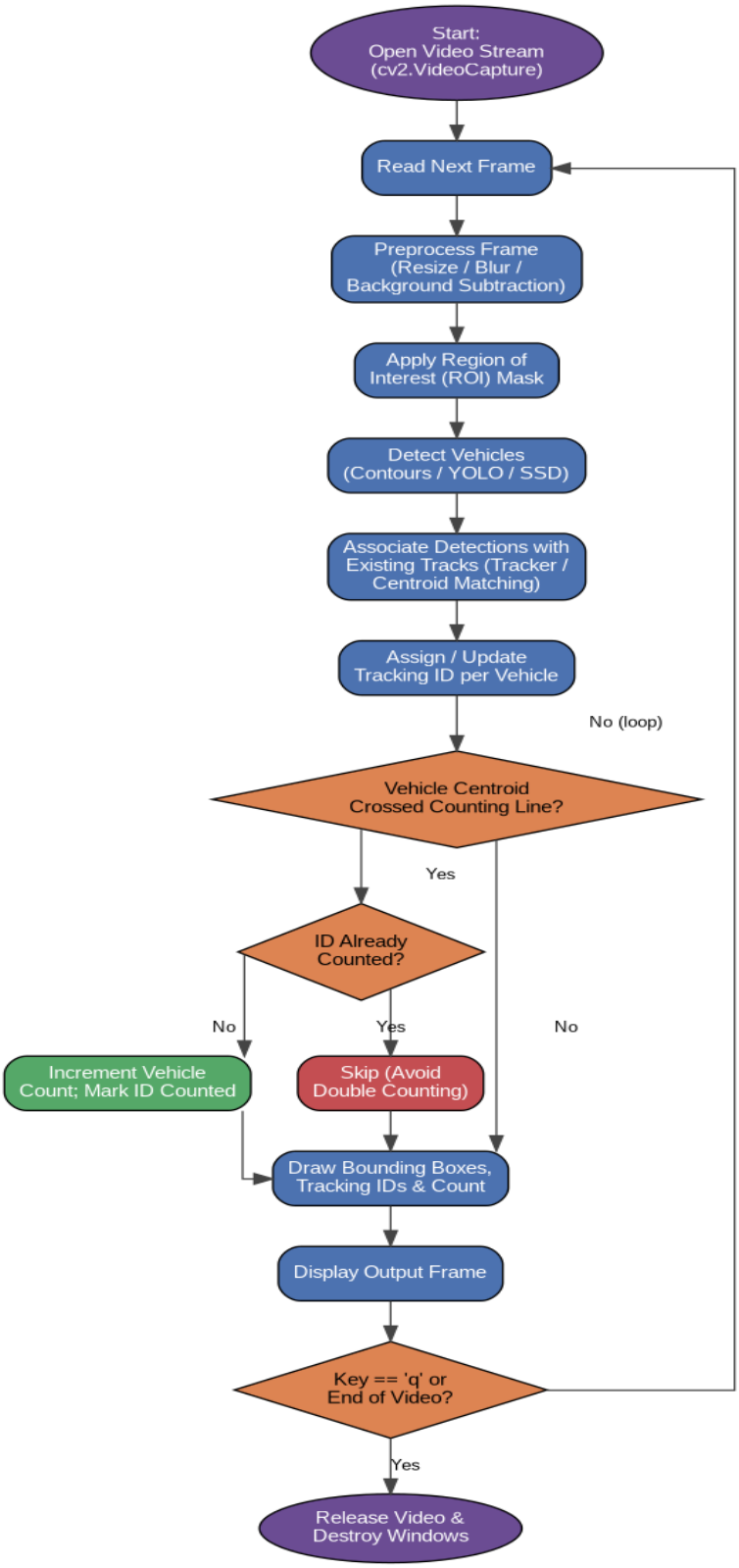

Fig. 2 — Vehicle Detection, Tracking & Counting Pipeline

Detailed Pseudocode

```
BEGIN VehicleDetectionTrackingCountingSystem

    // ---------- 1. INITIALIZATION ----------
    OPEN video stream: cap = VideoCapture(source)
    IF cap NOT opened THEN
        PRINT "Error: cannot access video source" AND EXIT
    END IF
    LOAD vehicle detector (background subtractor OR YOLO/SSD model)
    DEFINE region_of_interest (ROI) polygon/mask
    DEFINE counting_line: two points (Lx1, Ly1) - (Lx2, Ly2)
    INITIALIZE tracker = MultiObjectTracker()   // e.g., Centroid Tracker / SORT / DeepSORT
    INITIALIZE counted_ids = EMPTY SET          // prevents double counting
    vehicle_count = 0

    // ---------- 2. MAIN REAL-TIME LOOP ----------
    WHILE video stream has frames DO
        success, frame = cap.read()
        IF NOT success THEN
            BREAK   // end of video / stream error
        END IF

        // ---------- 3. PREPROCESSING ----------
        resized_frame = RESIZE(frame, target_width, target_height)
        blurred_frame = GAUSSIAN_BLUR(resized_frame, kernel_size)
        masked_frame = APPLY_ROI_MASK(blurred_frame, region_of_interest)

        // ---------- 4. VEHICLE DETECTION ----------
        IF using background_subtraction THEN
            fg_mask = background_subtractor.apply(masked_frame)
            fg_mask = MORPHOLOGICAL_OPS(fg_mask)      // remove noise, fill gaps
            contours = FIND_CONTOURS(fg_mask)
            detections = FILTER_BY_AREA(contours, min_area, max_area)
            bounding_boxes = GET_BOUNDING_RECTS(detections)
        ELSE   // deep-learning detector
            bounding_boxes, class_labels, scores = detector.predict(masked_frame)
            bounding_boxes = FILTER_BY_CLASS(bounding_boxes, class_labels,
                                             allowed = {car, truck, bus,
                                             bike})
            bounding_boxes = FILTER_BY_CONFIDENCE(bounding_boxes, scores,
        min_score)
        END IF

        // ---------- 5. MULTI-OBJECT TRACKING ----------
        tracked_objects = tracker.update(bounding_boxes)
        // tracked_objects = list of (track_id, x, y, w, h, centroid)
```

```
        // ---------- 6. COUNTING LOGIC (avoid double counting) ----------
        FOR EACH (track_id, x, y, w, h, centroid) IN tracked_objects DO
            IF CROSSED_LINE(centroid, previous_centroid[track_id], counting_line) THEN
                IF track_id NOT IN counted_ids THEN
                    vehicle_count = vehicle_count + 1
                    ADD track_id TO counted_ids
                END IF
            END IF
            previous_centroid[track_id] = centroid

            // ---------- 7. VISUALIZATION ----------
            DRAW_RECTANGLE(resized_frame, (x, y), (x+w, y+h), GREEN, thickness=2)
            DRAW_TEXT(resized_frame, "ID:" + track_id, position=(x, y-10))
        END FOR

        DRAW_LINE(resized_frame, counting_line, color=BLUE, thickness=2)
        DRAW_TEXT(resized_frame, "Count: " + vehicle_count, position=(20, 40))

        // ---------- 8. DISPLAY OUTPUT ----------
        DISPLAY(resized_frame, window_name = "Vehicle Detection, Tracking & Counting")

        // ---------- 9. LOOP CONTROL / EFFICIENT TERMINATION ----------
        IF KEY_PRESSED('q') OR END_OF_VIDEO THEN
            BREAK
        END IF
    END WHILE

    // ---------- 10. CLEANUP ----------
    cap.RELEASE()
    DESTROY_ALL_WINDOWS()
    PRINT "Final Vehicle Count: ", vehicle_count

END VehicleDetectionTrackingCountingSystem
```

Key Design Notes

- Avoiding double counting: each tracked vehicle carries a persistent track_id; once an ID crosses the counting line it is added to counted_ids, so subsequent frames for the same vehicle are not recounted.
- Handling multiple vehicles simultaneously: the multi-object tracker (Centroid Tracker / SORT / DeepSORT) associates every new detection with an existing or new track_id each frame, so an arbitrary number of vehicles are followed in parallel.
- Near real-time performance: an ROI mask restricts processing to the relevant part of the frame, background subtraction or a fast detector limits per-frame cost, and morphological filtering removes noise before contour analysis.
- Line-crossing logic: a vehicle is counted only when its centroid transitions from one side of the counting line to the other (comparing previous_centroid to the current centroid), not merely when it is near the line.
- Efficient loop termination: the WHILE loop exits cleanly on end-of-video or a keypress, always releasing the video handle and destroying display windows.